Module 6: Deep Learning & Neural Networks

Ultra-Comprehensive Guide to Building Intelligent Systems with TensorFlow and Keras

Introduction

Deep Learning is the cutting edge of artificial intelligence—enabling machines to learn complex patterns from raw data without hand-crafted features. Unlike traditional machine learning, deep learning automatically discovers the representations needed for feature detection and abstraction from raw input.

This module takes you from the fundamentals of neural networks (how a single neuron works) through to building complete deep learning systems with TensorFlow and Keras. You'll understand not just how to build neural networks, but **why** each component matters and **how** to tune them for peak performance.

By the end of this module, you'll be able to architect, train, and deploy neural networks for real-world problems—a skill that directly translates to industry roles building intelligent systems.

PART 1: INTRODUCTION TO NEURAL NETWORKS

Understanding the Perceptron: The Foundation

What is a Perceptron?

A **Perceptron** is the simplest neural network unit—a single neuron that mimics how biological neurons work.

Biological Inspiration:

- Biological neuron receives signals from dendrites
- Cell body integrates signals
- If threshold exceeded, neuron fires action potential

Artificial Neuron (Perceptron):

```
Input signals:  x₁, x₂, x₃, ...
Weights:        w₁, w₂, w₃, ...
Bias:           b
Activation:     a = w₁x₁ + w₂x₂ + w₃x₃ + b
Output:         y = f(a)  where f is activation function
```

Mathematical Formulation

```
z = Σ(wᵢ * xᵢ) + b  (Weighted sum + bias)
a = f(z)             (Apply activation function)
```

The Perceptron Decision Boundary

The perceptron creates a **linear decision boundary** in feature space:

```
w₁x₁ + w₂x₂ + b = 0  (Decision boundary in 2D)
```

Example:

```
Weights: w₁ = 1, w₂ = 1, b = -0.5
Equation: x + y - 0.5 = 0  or  y = -x + 0.5
```

Perceptron Learning Rule

Step 1: Initialize weights randomly (usually small values)

```
import numpy as np
np.random.seed(42)
w = np.random.randn(n_features) * 0.01  # Small random values
b = 0
```

Step 2: For each training sample:

```
z = w · x + b
a = activation(z)
error = y_actual - y_predicted

if error ≠ 0:
    w = w + learning_rate * error * x
    b = b + learning_rate * error
```

Step 3: Repeat until convergence

```
def perceptron_learning(X, y, learning_rate=0.01, epochs=100):
    w = np.zeros(X.shape[^1])
    b = 0

    for epoch in range(epochs):
        for x_sample, y_sample in zip(X, y):
            z = np.dot(w, x_sample) + b
            prediction = 1 if z &gt; 0 else 0  # Binary classification
            error = y_sample - prediction

            if error != 0:
```

```
                w = w + learning_rate * error * x_sample
                b = b + learning_rate * error

    return w, b
```

Limitation of Perceptron

XOR Problem: The perceptron cannot solve non-linearly separable problems.

```
XOR truth table:
x1  x2  output
0   0   0
0   1   1
1   0   1
1   1   0

Cannot draw single line separating 1s from 0s!
```

This limitation led to the **Multilayer Perceptron (MLP)** with hidden layers.

History of Neural Networks

Timeline:

1. **1943**: McCulloch & Pitts - First artificial neuron model
2. **1958**: Rosenblatt - Perceptron (simple learning algorithm)
3. **1969**: Minsky & Papert - Identified perceptron limitations (XOR problem)
4. **1986**: Rumelhart, Hinton, Williams - Backpropagation algorithm (revolutionary!)
5. **2012**: AlexNet wins ImageNet competition - Deep learning revolution begins
6. **2017**: Transformer architecture introduced (foundation for modern LLMs)

Activation Functions: The Non-Linearity Injection

Why Activation Functions Matter

Without activation functions, stacking layers doesn't help:

```
Multiple linear transformations = Single linear transformation
(w₁ * w₂ * x + b₁) + b₂ = w_combined * x + b_combined
```

Activation functions add non-linearity, allowing networks to learn complex patterns.

Sigmoid Activation Function

Mathematical Definition:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Properties:

- Output range: (0, 1)
- S-shaped smooth curve
- Derivative: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$
- Vanishing gradient problem for very negative/positive values

Use Case: Binary classification output layer (gives probability)

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

z = np.linspace(-10, 10, 100)
y = sigmoid(z)

plt.plot(z, y, 'b-', linewidth=2)
plt.xlabel("z")
plt.ylabel("σ(z)")
plt.title("Sigmoid Activation Function")
plt.grid(True, alpha=0.3)
plt.show()

# Output: S-shaped curve, smooth, differentiable
```

Problems with Sigmoid:

- Gradient near 0 when z is extreme (vanishing gradient)
- Slow convergence in deep networks
- Outputs not centered around 0 (complicates optimization)

ReLU (Rectified Linear Unit)

Mathematical Definition:

$$\text{ReLU}(z) = \max(0, z) = \begin{cases} 0 & amp; \text{if } z \quad lt; 0 \\ z & amp; \text{if } z \geq 0 \end{cases}$$

Properties:

- Output range: [0, ∞)

- Derivative: 1 if z > 0, else 0
- Simple, fast computation
- No vanishing gradient problem (when z > 0)

Use Case: Hidden layers (default choice in modern deep learning)

```
def relu(z):
    return np.maximum(0, z)

z = np.linspace(-5, 5, 100)
y = relu(z)

plt.plot(z, y, 'b-', linewidth=2)
plt.xlabel("z")
plt.ylabel("ReLU(z)")
plt.title("ReLU Activation Function")
plt.axhline(y=0, color='k', linestyle='-', alpha=0.3)
plt.axvline(x=0, color='k', linestyle='-', alpha=0.3)
plt.grid(True, alpha=0.3)
plt.show()

# Output: Linear for z &gt; 0, zero for z &lt; 0
```

Advantages:

- Prevents vanishing gradient (constant gradient when z > 0)
- Fast computation
- Better training convergence

Disadvantage - Dying ReLU:

- If neuron outputs always ≤ 0, gradient = 0 forever (dead neuron)
- Neuron never updates weights again

Leaky ReLU

Mathematical Definition:

$$\text{Leaky ReLU}(z) = \begin{cases} \alpha z & amp; \text{if } z \quad lt; 0 \\ z & amp; \text{if } z \geq 0 \end{cases}$$

Where α is small positive value (typically 0.01)

Purpose: Prevent dying ReLU problem by allowing small gradient when z < 0

```
def leaky_relu(z, alpha=0.01):
    return np.where(z &gt; 0, z, alpha * z)

z = np.linspace(-5, 5, 100)
y = leaky_relu(z)
```

```
plt.plot(z, y, 'b-', linewidth=2)
plt.xlabel("z")
plt.ylabel("Leaky ReLU(z)")
plt.title("Leaky ReLU Activation Function")
plt.grid(True, alpha=0.3)
plt.show()

# Output: Small slope for z &lt; 0, linear for z &gt; 0
```

Benefits:

- Addresses dying ReLU problem
- Still maintains computational efficiency

Tanh (Hyperbolic Tangent)

Mathematical Definition:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = \frac{2}{1 + e^{-2z}} - 1$$

Properties:

- Output range: (-1, 1)
- Derivative: $\tanh'(z) = 1 - \tanh^2(z)$
- Stronger gradients than sigmoid
- Zero-centered (better for optimization)

Use Case: Recurrent neural networks (LSTMs), sometimes hidden layers

```
def tanh(z):
    return np.tanh(z)

z = np.linspace(-5, 5, 100)
y = tanh(z)

plt.plot(z, y, 'b-', linewidth=2)
plt.xlabel("z")
plt.ylabel("tanh(z)")
plt.title("Tanh Activation Function")
plt.axhline(y=0, color='k', linestyle='-', alpha=0.3)
plt.axvline(x=0, color='k', linestyle='-', alpha=0.3)
plt.grid(True, alpha=0.3)
plt.show()

# Output: S-shaped curve, range (-1, 1)
```

Softmax Activation Function

Mathematical Definition:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Properties:

- Output range: (0, 1) for each output
- Sum of all outputs = 1 (probability distribution)
- Maximum value receives most probability mass

Use Case: Multi-class classification output layer (gives probability for each class)

Example:

```
def softmax(z):
    exp_z = np.exp(z - np.max(z))  # Subtract max for numerical stability
    return exp_z / np.sum(exp_z)

# Logits from neural network
logits = np.array([2.0, 1.0, 0.1])
probabilities = softmax(logits)
print(probabilities)  # Output: [0.665, 0.244, 0.090]
# Class 0 has 66.5% probability, class 1 has 24.4%, class 2 has 9.0%
```

Why subtract max? Prevents numerical overflow when computing e^{z_i}

Activation Function Comparison Table

Function	Range	Use Case	Problem	Derivative
Sigmoid	(0,1)	Output layer (binary)	Vanishing gradient	σ(1-σ)
ReLU	[0,∞)	Hidden layers	Dying ReLU	0 or 1
Leaky ReLU	(-∞,∞)	Hidden layers	More gradient	α or 1
Tanh	(-1,1)	RNNs, hidden layers	Slower than ReLU	$1-\tanh^2$
Softmax	(0,1) sum=1	Output (multiclass)	Not applicable	Complex

Gradient Descent: The Optimization Engine

What is Gradient Descent?

Goal: Find weights that minimize loss function

Analogy: Hiking down a mountain in the fog

- Can't see destination (global minimum)
- Only feel slope under feet (gradient)
- Take steps downhill
- Eventually reach valley

Mathematical Foundation

Loss Function: $L(w, b) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$

We want to find w^* that minimizes L:

$$w^* = \arg\min_w L(w)$$

Gradient: Vector of partial derivatives

$$\nabla L = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \ldots, \frac{\partial L}{\partial w_n}\right]$$

Intuition: Gradient points toward steepest increase; we go opposite direction

Gradient Descent Update Rule

```
w_new = w_old - learning_rate * ∇L
b_new = b_old - learning_rate * (∂L/∂b)
```

Step size = learning_rate × gradient magnitude

Step-by-Step Gradient Descent

```
import numpy as np
import matplotlib.pyplot as plt

def compute_gradient(w, X, y):
    """Compute gradient of loss with respect to w"""
    predictions = np.dot(X, w)
    error = predictions - y
    gradient = np.dot(X.T, error) / len(y)
    return gradient

def gradient_descent(X, y, learning_rate=0.01, iterations=100):
    w = np.zeros(X.shape[^1])
    losses = []

    for i in range(iterations):
```

```
        # Compute loss
        predictions = np.dot(X, w)
        loss = np.mean((predictions - y) ** 2)
        losses.append(loss)

        # Compute gradient
        gradient = compute_gradient(w, X, y)

        # Update weights
        w = w - learning_rate * gradient

        if i % 10 == 0:
            print(f"Iteration {i}: Loss = {loss:.4f}")

    return w, losses

# Visualize loss decreasing
plt.plot(losses)
plt.xlabel("Iteration")
plt.ylabel("Loss")
plt.title("Gradient Descent: Loss Converging to Minimum")
plt.show()
```

Challenges with Gradient Descent

1. Learning Rate Selection:

- **Too small**: Convergence too slow (millions of iterations needed)
- **Too large**: Overshoots minimum (divergence or oscillation)

```
Small LR:  w ← w - 0.0001 * ∇L  (tiny steps, slow)
Large LR:  w ← w - 1.0 * ∇L     (huge jumps, misses optimum)
Optimal:   w ← w - 0.01 * ∇L    (Goldilocks zone)
```

2. Local Minima (in non-convex problems):

- Gradient descent might get stuck in local minimum
- Not the global best solution

3. Saddle Points:

- Points where gradient = 0 but not a minimum
- Can trap optimization

Learning Rate Tuning Strategies

Manual Tuning:

```
# Start with standard value
learning_rate = 0.01
```

```
# If loss oscillates: Too high, reduce
learning_rate = 0.001

# If convergence too slow: Too low, increase
learning_rate = 0.1

# Find sweet spot through experimentation
```

Learning Rate Schedules:

```
def learning_rate_schedule(epoch, initial_lr=0.01):
    # Decrease LR as training progresses
    if epoch &lt; 10:
        return initial_lr
    elif epoch &lt; 20:
        return initial_lr * 0.5
    else:
        return initial_lr * 0.1

# Usage in training loop
for epoch in range(30):
    current_lr = learning_rate_schedule(epoch)
    # Train with current_lr
```

Optimization Functions: Beyond Basic Gradient Descent

Problems with Standard Gradient Descent

Oscillation in narrow valleys:

```
High curvature in one direction
Gradient descent zigzags
Slow convergence
```

SGD (Stochastic Gradient Descent)

Idea: Update on single sample (not entire batch)

```
def sgd(X, y, learning_rate=0.01, epochs=10):
    w = np.zeros(X.shape[^1])

    for epoch in range(epochs):
        for x_sample, y_sample in zip(X, y):
            prediction = np.dot(w, x_sample)
            error = y_sample - prediction
            gradient = -error * x_sample  # Gradient for single sample
            w = w + learning_rate * gradient  # Frequent updates

    return w
```

Advantages:

- Frequent updates → faster convergence
- Can escape local minima (noise helps)

Disadvantages:

- Noisy updates (high variance)
- Oscillation due to noise

Mini-batch Gradient Descent

Idea: Update on small batch (32, 64, 128 samples)

```
Balance between efficiency and stability:
- Batch size = 1: Very noisy (SGD)
- Batch size = 32: Good noise-stability tradeoff
- Batch size = entire dataset: No noise (Batch GD)
```

Momentum Optimizer

Idea: Build up velocity like rolling ball downhill

$$v_t = \beta v_{t-1} + (1-\beta)\nabla L$$

$$w = w - \text{learning_rate} \times v_t$$

Where β ≈ 0.9 (momentum coefficient)

Effect:

- Accumulates gradients over time
- Accelerates in consistent directions
- Dampens oscillations

```
def momentum_optimizer(gradients, learning_rate=0.01, momentum=0.9):
    velocity = 0  # Initialize velocity

    for gradient in gradients:
        velocity = momentum * velocity + gradient
        weight_update = -learning_rate * velocity

    return weight_update
```

Analogy: Ball rolling downhill picks up speed and momentum, smooths out terrain

Adam Optimizer (Adaptive Moment Estimation)

Most popular optimizer in deep learning

Idea: Adapts learning rate per parameter based on gradient history

```
def adam(gradient, learning_rate=0.001, beta1=0.9, beta2=0.999, epsilon=1e-8):
    """
    beta1: Momentum coefficient for first moment (mean)
    beta2: Momentum coefficient for second moment (variance)
    epsilon: Small number to prevent division by zero
    """
    m = beta1 * m_prev + (1 - beta1) * gradient  # First moment (mean)
    v = beta2 * v_prev + (1 - beta2) * gradient**2  # Second moment (variance)

    m_hat = m / (1 - beta1**t)  # Bias correction
    v_hat = v / (1 - beta2**t)  # Bias correction

    weight_update = -learning_rate * m_hat / (np.sqrt(v_hat) + epsilon)

    return weight_update
```

Why Adam Works:

1. **Momentum** (first moment) - accelerates in consistent directions
2. **Adaptive learning rates** (second moment) - per-parameter adjustment
3. **Bias correction** - better initial estimates

Example: In high-curvature directions, Adam reduces learning rate; in gentle directions, it increases it

Optimizer Comparison

Optimizer	Convergence Speed	Stability	When to Use
SGD	Medium	Low	When you need full control
SGD+Momentum	Fast	Medium	Classic choice
Adam	Very Fast	High	Default for most tasks
RMSprop	Fast	High	When Adam not available
Adadelta	Fast	High	No learning rate needed

Recommendation: Start with Adam (works in 90% of cases)

Backpropagation and Chain Rule: How Networks Learn

Forward Pass vs Backward Pass

Forward Pass (Inference):

```
Input → Layer1 → Layer2 → Layer3 → Output
Compute predictions
```

Backward Pass (Learning):

```
Loss at Output ← Layer3 ← Layer2 ← Layer1 ← Input
Propagate error back
Compute gradients for each weight
```

The Chain Rule in Calculus

Simple Chain Rule:

$$\frac{df}{dx} = \frac{df}{du} \cdot \frac{du}{dx}$$

Interpretation: To find how input x affects output f, multiply the effect of x on intermediate u and u's effect on f

Example with Neural Network:

```
Loss L = (y_pred - y_true)²
y_pred = σ(z)
z = w·x + b

How does L change with w?
∂L/∂w = (∂L/∂y_pred) · (∂y_pred/∂z) · (∂z/∂w)
      = 2(y_pred - y_true) · σ'(z) · x
```

Backpropagation Step-by-Step

Simple 2-layer network:

```
Input x → Dense Layer → ReLU → Dense Layer → Sigmoid → Output ŷ
       w1,b1         w2,b2

Loss = (ŷ - y)²
```

Backward Pass:

Step 1: Output layer gradient

```
∂Loss/∂ŷ = 2(ŷ - y)
```

Step 2: Before sigmoid gradient

```
∂Loss/∂z_out = (∂Loss/∂ŷ) · (∂ŷ/∂z_out)
             = 2(ŷ - y) · σ'(z_out)
```

Step 3: Gradients for w2, b2

```
∂Loss/∂w2 = (∂Loss/∂z_out) · (∂z_out/∂w2)
          = (∂Loss/∂z_out) · hidden_output

∂Loss/∂b2 = (∂Loss/∂z_out) · (∂z_out/∂b2)
          = (∂Loss/∂z_out) · 1
```

Step 4: Propagate back to hidden layer

```
∂Loss/∂hidden = (∂Loss/∂z_out) · (∂z_out/∂hidden)
              = (∂Loss/∂z_out) · w2
```

Step 5: Hidden layer gradient (through ReLU)

```
∂Loss/∂z_hidden = (∂Loss/∂hidden) · (∂hidden/∂z_hidden)
                = (∂Loss/∂hidden) · ReLU'(z_hidden)
```

Step 6: Gradients for w1, b1

```
∂Loss/∂w1 = (∂Loss/∂z_hidden) · x
∂Loss/∂b1 = (∂Loss/∂z_hidden)
```

Step 7: Weight updates

```
w1_new = w1 - learning_rate · (∂Loss/∂w1)
w2_new = w2 - learning_rate · (∂Loss/∂w2)
b1_new = b1 - learning_rate · (∂Loss/∂b1)
b2_new = b2 - learning_rate · (∂Loss/∂b2)
```

Vanishing Gradient Problem

When backpropagating through many layers with sigmoid:

```
∂L/∂w1 = ∂L/∂ŷ · ∂ŷ/∂z_layer3 · ∂z_layer3/∂z_layer2 · ... · ∂z_layer1/∂w1
       = ε · σ'(z3) · σ'(z2) · σ'(z1) · x

Sigmoid derivative max = 0.25
Gradient = 0.25 * 0.25 * 0.25 * ... (very small!)
```

Result: Early layers get tiny gradients → learn very slowly

Solutions:

1. Use ReLU (derivative = 1 when z > 0)
2. Initialize weights carefully
3. Use batch normalization
4. Use LSTM for sequences

Introduction to TensorFlow and Keras

What is TensorFlow?

TensorFlow is Google's open-source deep learning framework

Key Components:

- Low-level API: Tensor operations
- High-level API: Keras (easy neural networks)
- Optimized for GPU training

```
import tensorflow as tf

# Check TensorFlow version
print(tf.__version__)  # Output: 2.x

# Create tensor
tensor = tf.constant([[1, 2, 3], [4, 5, 6]])
print(tensor)

# Tensor operations
result = tf.matmul(tensor, tf.transpose(tensor))
print(result)
```

What is Keras?

Keras is high-level API on top of TensorFlow

Philosophy: User-friendly, modular, extensible

```
from tensorflow import keras

# Simple model in Keras
model = keras.Sequential([
    keras.layers.Dense(128, activation='relu', input_shape=(784,)),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])
```

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=10, batch_size=32)
```

Fully Connected Layers and Weight Initialization

Fully Connected Layer

Definition: Every input connected to every neuron

```
Input layer (4 neurons) → Hidden layer (3 neurons) → Output (1 neuron)
Total connections = 4*3 + 3*1 = 15
Total parameters = 15 weights + 3+1 biases = 19
```

Weight Initialization Strategies

Problem: Wrong initialization → poor convergence

All zeros (BAD):

```
w = np.zeros((input_size, output_size))
# All neurons compute same thing → complete redundancy
```

Too large (BAD):

```
w = np.random.randn(input_size, output_size) * 100
# Activations → extreme values
# Gradients → vanishing/exploding
```

Xavier/Glorot Initialization (GOOD):

```
import numpy as np

def xavier_init(input_size, output_size):
    limit = np.sqrt(6 / (input_size + output_size))
    return np.random.uniform(-limit, limit, (input_size, output_size))

# Idea: Maintain variance of activations through layers
# Works well with tanh, sigmoid
```

He Initialization (BETTER for ReLU):

```
def he_init(input_size, output_size):
    std = np.sqrt(2 / input_size)
    return np.random.randn(input_size, output_size) * std

# Idea: Account for ReLU killing half the activations
# Variance larger to compensate
```

In Keras:

```
keras.layers.Dense(128, kernel_initializer='he_normal', activation='relu')
```

Regularization: Preventing Overfitting

L1 and L2 Regularization

Problem: Weights grow unbounded → overfitting

Solution: Add penalty to loss function

L2 Regularization (Ridge):

$$Loss = MSE + \lambda \sum_i w_i^2$$

Effect: Encourages small weights (smooth function)

```
keras.layers.Dense(128, kernel_regularizer=keras.regularizers.l2(0.01))
```

L1 Regularization (Lasso):

$$Loss = MSE + \lambda \sum_i |w_i|$$

Effect: Zeros out unimportant weights (feature selection)

Dropout

Idea: Randomly turn off neurons during training

```
During training: With probability p, set neuron output = 0
During inference: Use all neurons (scaled)
```

Why it works:

- Prevents co-adaptation of neurons

- Ensemble effect (different networks each batch)
- Forces neurons to learn robust features

```
model = keras.Sequential([
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dropout(0.2),  # Turn off 20% of neurons
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(10, activation='softmax')
])
```

Batch Normalization

Idea: Normalize activations to have mean=0, std=1

$$x_{norm} = \frac{x - mean}{std}$$

Benefits:

- Stabilizes training
- Allows higher learning rates
- Reduces internal covariate shift
- Acts as regularizer

```
model = keras.Sequential([
    keras.layers.Dense(128, activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(10, activation='softmax')
])
```

Cross Entropy Loss

Binary Cross Entropy

For binary classification (output ∈ [0,1]):

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Interpretation:

- When y=1: Loss = -log(ŷ) (penalizes low predicted probability)
- When y=0: Loss = -log(1-ŷ) (penalizes high predicted probability)

```
# Model outputs probability
y_true = 1
```

```
y_pred = 0.8

loss = -(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))
# = -(1 * log(0.8) + 0 * log(0.2))
# = -log(0.8) ≈ 0.223
```

Categorical Cross Entropy

For multi-class (output is probability distribution):

$$L = -\sum_{i=1}^{K} y_i \log(\hat{y}_i)$$

```
# One-hot encoded true label
y_true = [1, 0, 0]  # Class 0
y_pred = [0.7, 0.2, 0.1]  # Probabilities from softmax

loss = -(1 * log(0.7) + 0 * log(0.2) + 0 * log(0.1))
     = -log(0.7) ≈ 0.357
```

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',  # Multi-class
    metrics=['accuracy']
)
```

TensorFlow Graphs and Eager Execution

Eager Execution (Default in TF 2.x)

Operations execute immediately:

```
import tensorflow as tf

# Immediate execution
a = tf.constant([[1, 2], [3, 4]])
b = tf.constant([[5, 6], [7, 8]])
c = tf.matmul(a, b)
print(c)  # Output printed immediately
```

TensorFlow Graphs (Advanced)

Build computational graph, then execute:

```
@tf.function
def compute(x):
    return x ** 2 + 2 * x + 1
```

```
# Graph created on first call
result = compute(tf.constant(3.0))
print(result)  # Executes optimized graph
```

References

1. Goodfellow, I., Bengio, Y., & Courville, A. (2023). Deep Learning. MIT Press.
2. LeCun, Y., Bengio, Y., & Hinton, G. (2023). Deep learning. Nature, 521(7553), 436-444.
3. Kingma, D. P., & Ba, J. (2023). Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980.
4. Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (2023). Learning representations by back-propagating errors. Nature, 323(6088), 533-536.
5. Chollet, F. (2023). Deep Learning with Python (2nd ed.). Manning Publications.
6. TensorFlow Documentation. (2024). Retrieved from https://www.tensorflow.org/
7. Keras Documentation. (2024). Retrieved from https://keras.io/
8. Andrew Ng. (2024). Deep Learning Specialization. Coursera. Retrieved from https://www.coursera.org/
9. Stanford CS231n: CNN for Visual Recognition. (2024). Retrieved from http://cs231n.stanford.edu/
10. FastAI: Practical Deep Learning. (2024). Retrieved from https://course.fast.ai/
[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19]

⁂

1. https://arxiv.org/html/2408.10006v1
2. https://nlp.stanford.edu/projects/glove/
3. https://towardsdatascience.com/word2vec-glove-and-fasttext-explained-215a5cd4c06f/
4. https://www.ibm.com/think/topics/word-embeddings
5. https://www.geeksforgeeks.org/nlp/word-embeddings-in-nlp/
6. https://www.linkedin.com/pulse/mastering-word-embeddings-nlp-comprehensive-journey-through-sachin-imekc
7. https://www.geeksforgeeks.org/computer-vision/transfer-learning-for-computer-vision/
8. https://journals.plos.org/plosone/article?id=10.1371%2Fjournal.pone.0300767
9. https://nlp2024.jeju.ai/en/week04/index.html
10. https://arxiv.org/abs/2406.10050
11. https://www.geeksforgeeks.org/machine-learning/backpropagation-in-neural-network/
12. https://www.oncologyradiotherapy.com/articles/mcripunet-a-modified-unet-architecture-for-semantic--segmentation-of-images-for-breast-cancer-detection.pdf
13. https://towardsdatascience.com/neural-networks-backpropagation-by-dr-lihi-gur-arie-27be67d8fdce/
14. https://en.wikipedia.org/wiki/Backpropagation
15. https://pub.towardsai.net/backpropagation-2eeb25201095
16. https://viso.ai/deep-learning/convolution-operations/

17. https://www.sciencedirect.com/topics/computer-science/long-short-term-memory-network

18. https://www.linkedin.com/pulse/neural-network-chain-rule-understanding-algorithm-deep-doug-rose-maaie

19. https://ijcat.com/archieve/volume13/issue6/ijcatr13061003.pdf